

Prezentarea 3

Prof. Coordonator: Ș.I.dr.ing.Stelian-Nicolae NICOLĂ

Mentor: Pătrașcu Hary

Resurse de învățarea

Colegul vostru a creat niște notebook-uri în python pentru a învăța programare mai ușor. Le puteți găsi în folder-ul prezentărilor de pe github. După ce le downloadați trebuie să accesați site-ul jupyter.org și să uploadați notebook-ul.

 JupyterLab

FileViewSettingsHelp

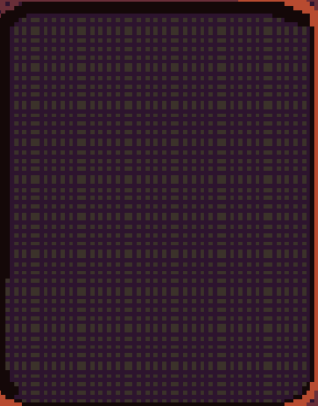
Files

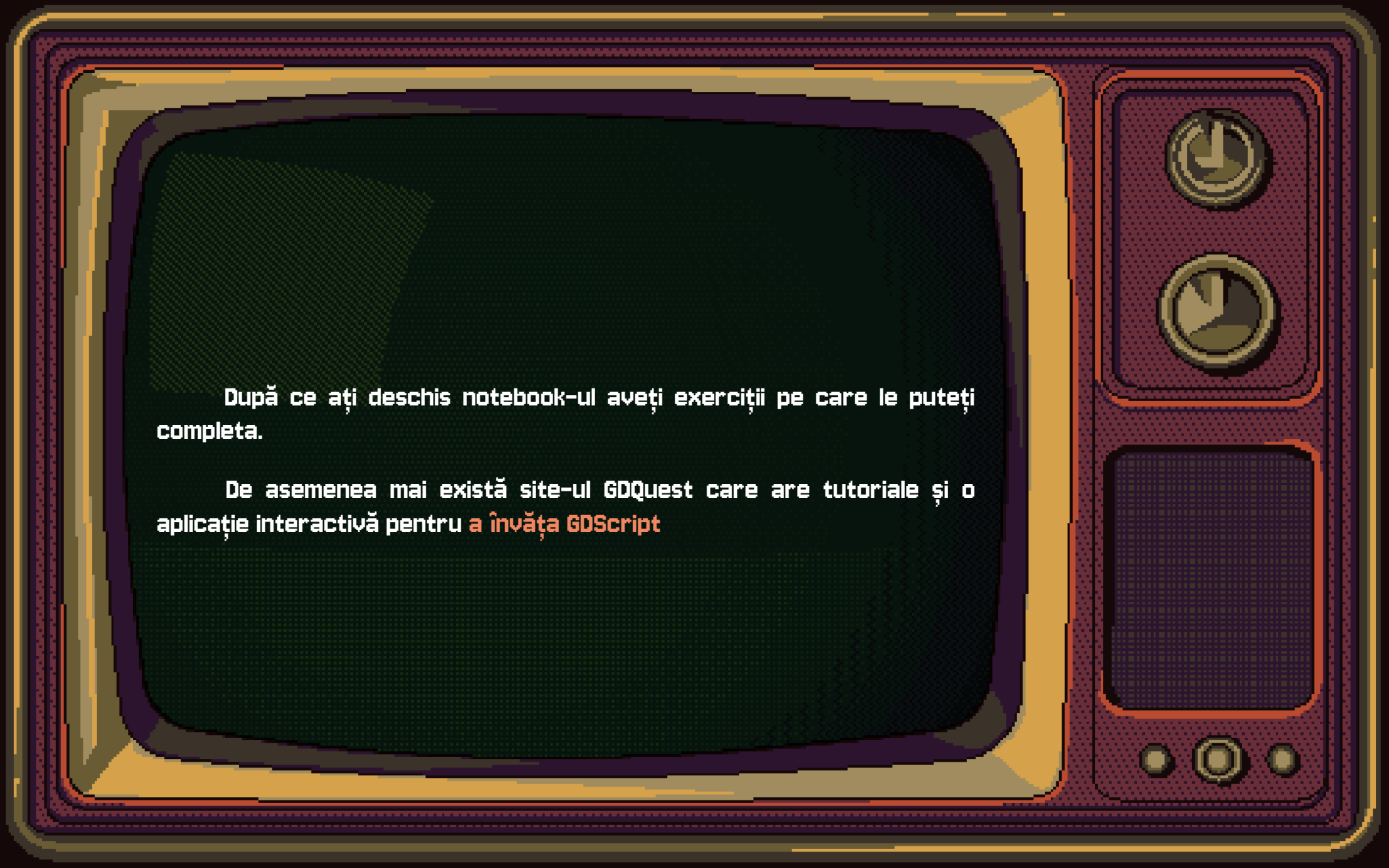
Select items to perform actions on them.

▼ New ⬆ Upload ↻

/ notebooks /

Name	Last Modified
 cpp-third-party-libs.ipynb	17 days ago
 cpp-tiny-ray-tracer.ipynb	17 days ago
 cpp.ipynb	17 days ago
 Intro.ipynb	17 days ago
 Lorenz.ipynb	17 days ago
 presentation_2_3_challenge_notebook.ipynb	7 seconds ago
 r.ipynb	17 days ago
 sqlite.ipynb	17 days ago





După ce ați deschis notebook-ul aveți exerciții pe care le puteți completa.

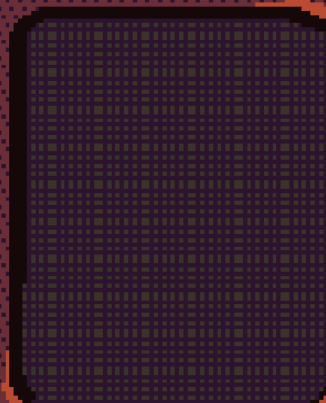
De asemenea mai există site-ul GDQuest care are tutoriale și o aplicație interactivă pentru **a învăța GDScript**

Continuăm de unde am rămas...

În prezentarea anterioară, am reușit să facem jucătorul să interacționeze cu diferite unelte. Acum vrem ca jucătorul să poată interacționa cu lumea din jur.

Haideți să gândim problema împreună, ce pași credeți că trebuie să luăm pentru a realiza asta?

În primul, rând, pentru a rezolva o problemă, trebuie să punem întrebările corecte.



Începând cu întrebările...

- Cum facem două noduri să comunice între ele? (player și nivel)
- Semnale(signals)
- Odată ce am trimis datele la nivel, cum facem ca nivelul să reacționeze?
- Callbacks(metode care răspund la semnalul creat)

Exercițiu:

- Atașați un script nodului 'Level'.
- Creați un semnal 'tool_used' în scriptul player care primește ca parametru o variabilă de tipul 'Tool'.
- La sfârșitul animațiilor de tool usage, emiteți semnalul cu `<signal_name>.emit(arg0, arg1, ...)`.
- Conectați semnalul creat în player cu nodul level.(există două moduri pentru asta)



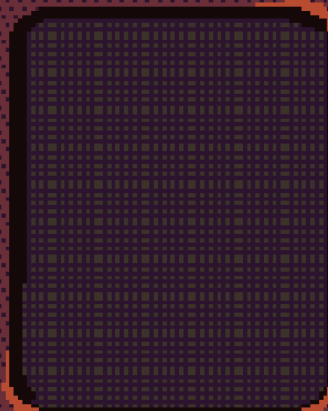
```
# signal example(observer pattern)
signal tool_used(tool: Tool)
```

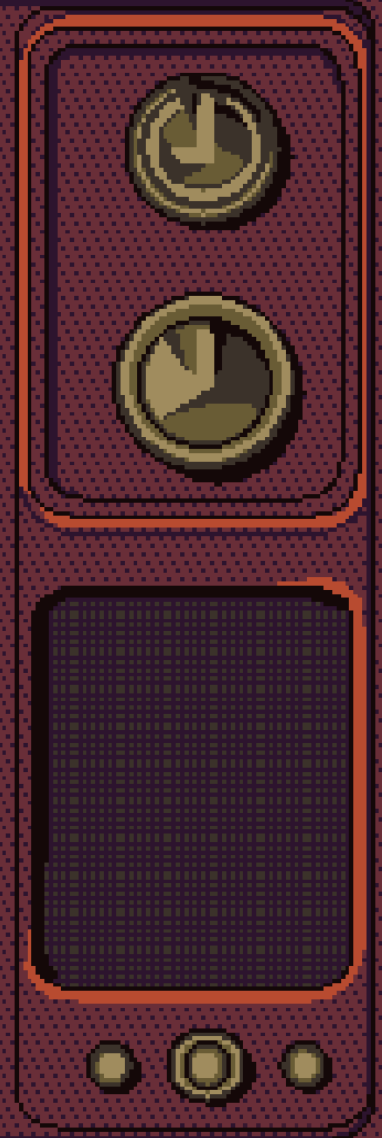
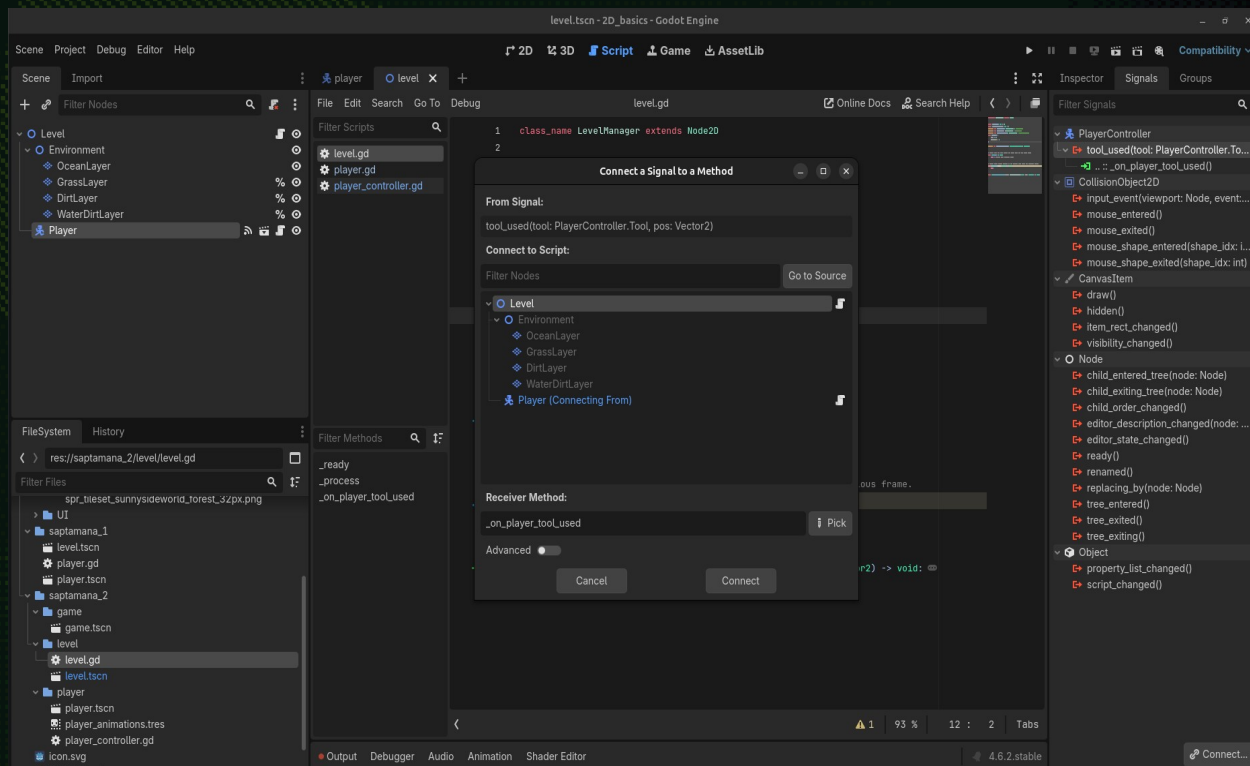


`await character_animated_sprite.animation_finished`
`await tools_animated_sprite.animation_finished`
`tool_used.emit(_current_tool)`

Conectarea semnalelor la noduri

- Selectați nodul care conține semnalul creat(Playerul)
- În partea dreaptă unde este inspectorul, aveți un tab numit signals
- În tabul signals o să apară semnalul creat de voi
- Dați double click pe semnal și veți avea un meniu unde puteți să numiți semnalul și să alegeți nodul cu care vreți să îl conectați(un semnal poate fi conectat cu mai multe noduri dacă vreți)
- În scriptul nodului cu care ați conectat semnalul o să apară o metodă de callback care va fi apelată de fiecare dată când emiteți semnalul





O altă variantă de a conecta semnale...

- Adăugați o referință la player în nodul level
- Accesați referința, numele semnalului și apelați metoda connect care accepta numele unei metode(callback-ul)
- Conectarea semnalelor se face de obicei în metoda `_ready()`



```
16  @onready var player: PlayerController = $Player
17
18  # Called when the node enters the scene tree for the first time.
19  ▾ func _ready() -> void:
20    > player.tool_used.connect(_on_player_tool_used)
21    |
```

Metode anonime

- În loc să creem o metodă în script, putem să atribuim direct o metodă anonimă:

```
18 # Called when the node enters the scene tree for the first time.  
19 ▾ func _ready() -> void:  
20 ▾ ▹ player.tool_used.connect(func():  
21   ▹   print("the tool was used")  
22   ▹ )  
23
```


Filter Methods

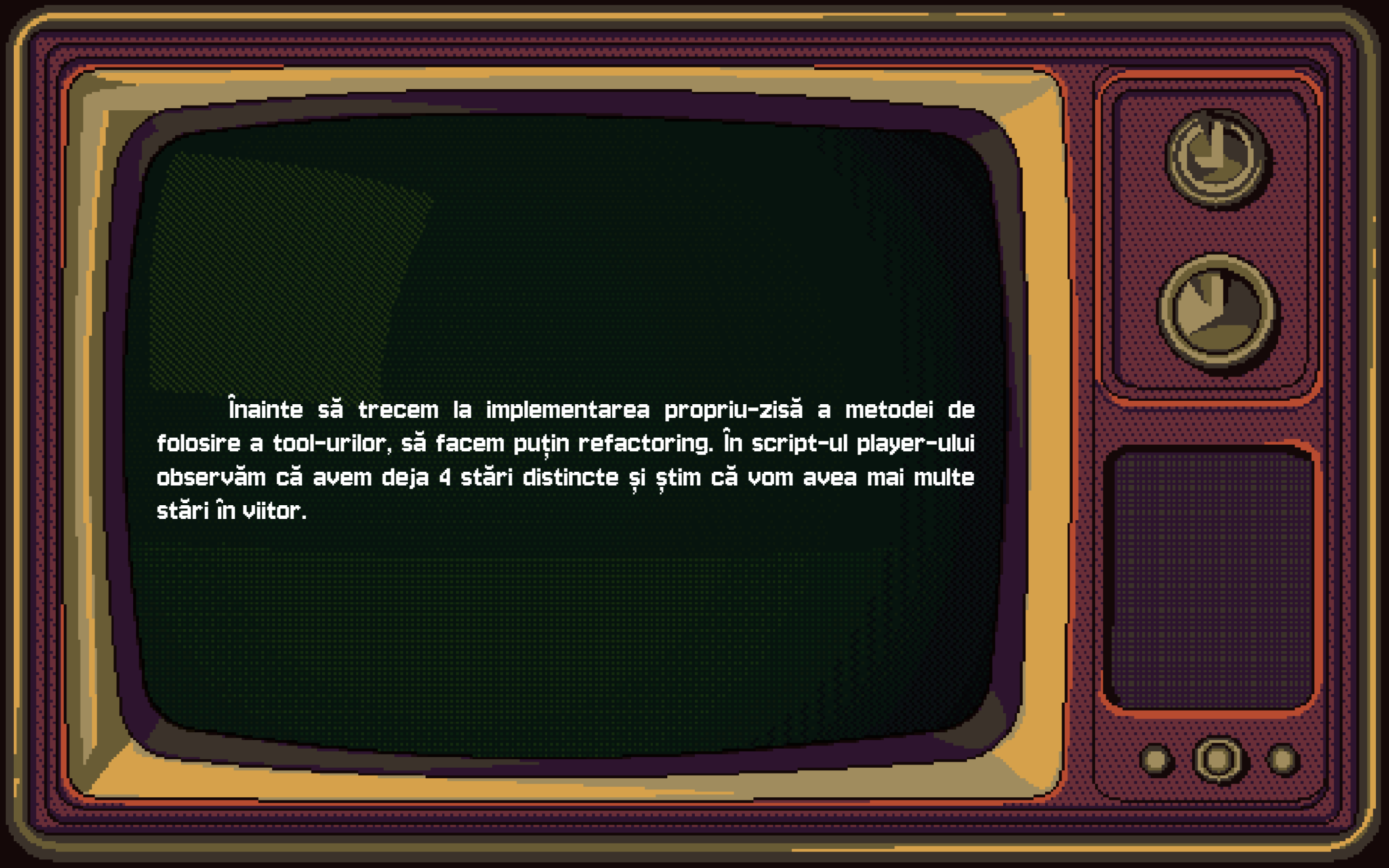
_ready
_process
_on_player_tool_used

```
16 @onready var player: PlayerController = $Player
17
18 # Called when the node enters the scene tree for the first time.
19 func _ready() -> void:
20     player.tool_used.connect(func(tool: PlayerController.Tool):
21         print("%s tool was used" % tool)
22     )
23
24
25 # Called every frame. 'delta' is the elapsed time since the previous frame.
26 func _process(delta: float) -> void:
27     pass
28
29
30 func _on_player_tool_used(tool: PlayerController.Tool, pos: Vector2) -> void:
44
```

1 93 % 22 :

Godot Engine v4.6.2.stable.official.71f334935 - <https://godotengine.org>
OpenGL API 3.3.0 NVIDIA 580.159.03 - Compatibility - Using Device: NVIDIA - NVIDIA GeForce RTX 5060 Ti

0 tool was used

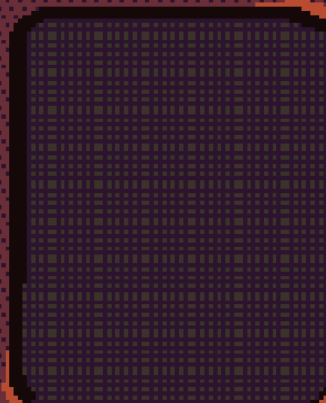


Înainte să trecem la implementarea propriu-zisă a metodei de folosire a tool-urilor, să facem puțin refactoring. În script-ul player-ului observăm că avem deja 4 stări distincte și știm că vom avea mai multe stări în viitor.

Automate cu stări finite(Finite state machine)

FSN este un design pattern folosit des în jocuri video și în alte domenii software/hardware pentru a gestiona mai ușor flow-ul comportamentului/datelor. În jocuri video sunt folosite în general pentru AI, npc-uri sau caracterul principal controlat de jucător.

Design pattern-urile sunt soluții găsite de programatori pe parcursul istoriei programării pentru probleme întâlnite des în ingineria software.



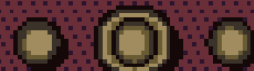
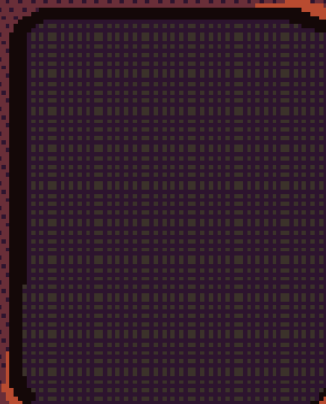
Vom face o implementare naivă pentru început și în viitor o să creăm un FSN mai complex(la modulul 3D).

Ce problemă rezolvă un FSN?

Scapă de cascadele infinite de if elif care apar inevitabil cu cât adăugam mai multă logică la scriptul player-ului.

Putem adăuga stări noi fără să trebuiască să modificăm stări anterioare.

Izolează codul stărilor(cum am făcut inițial cu metode).

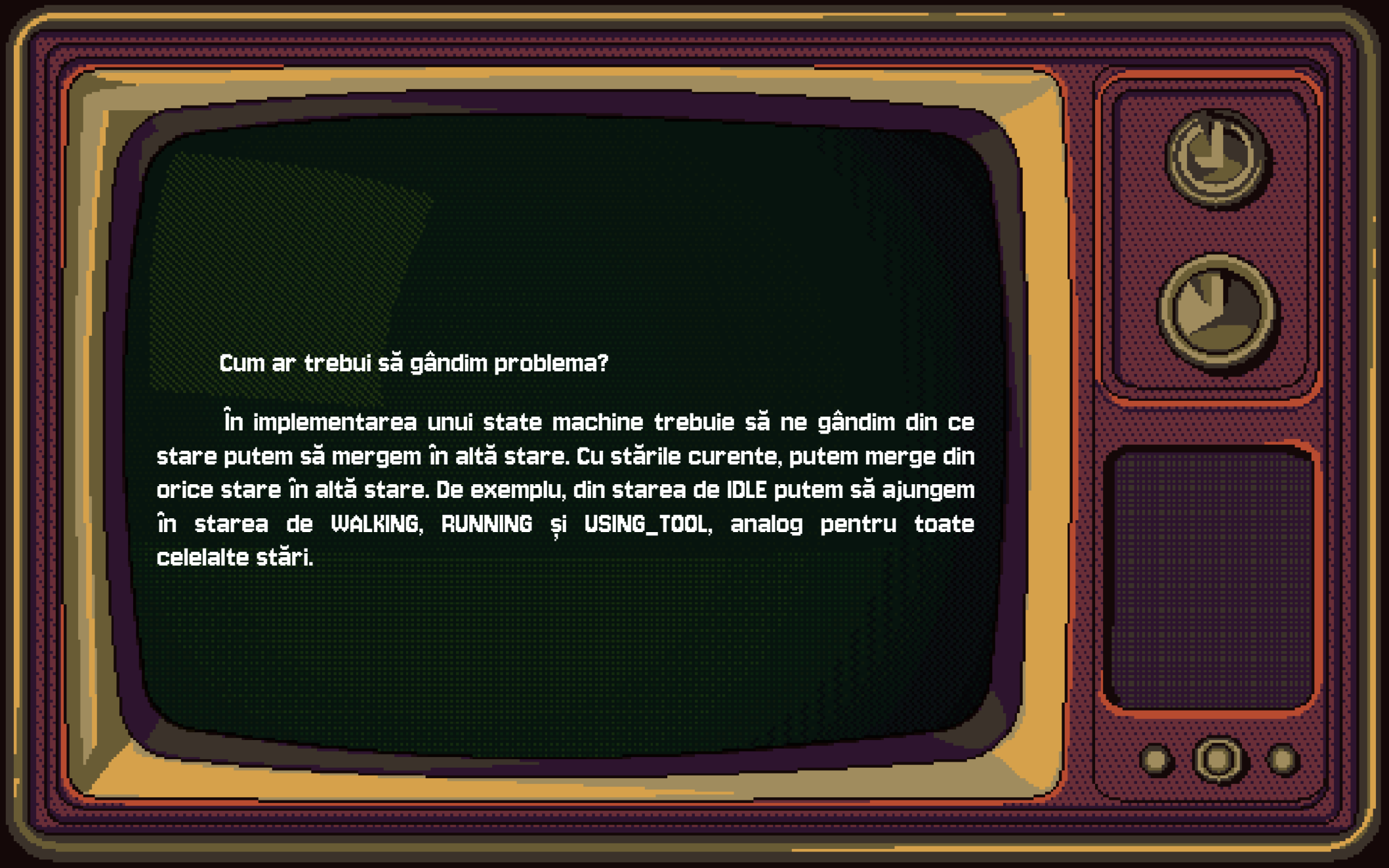


Implementarea unui FSN simplu

Exercițiu: creați un enum cu numele 'State' care să conțină stările: IDLE, WALKING, RUNNING, USING_TOOL.

Creați o variabilă `_current_state` de tipul State care se află în starea inițială IDLE.

```
32
33  ▾ enum State {
34    >| IDLE,
35    >| WALKING,
36    >| RUNNING,
37    >| USING_TOOL
38  }
39  @export var _current_state: State
```



Cum ar trebui să gândim problema?

În implementarea unui state machine trebuie să ne gândim din ce stare putem să mergem în altă stare. Cu stările curente, putem merge din orice stare în altă stare. De exemplu, din starea de IDLE putem să ajungem în starea de WALKING, RUNNING și USING_TOOL, analog pentru toate celelalte stări.

Match keyword

O să avem nevoie de un match statement (switch în alte limbaje de programare) pentru următorul exercițiu. Match funcționează ca o structură de control (if/elif). Compară o variabilă, și se duce pe branch-ul care trebuie. Aveți mai multe informații în documentația link-uită.

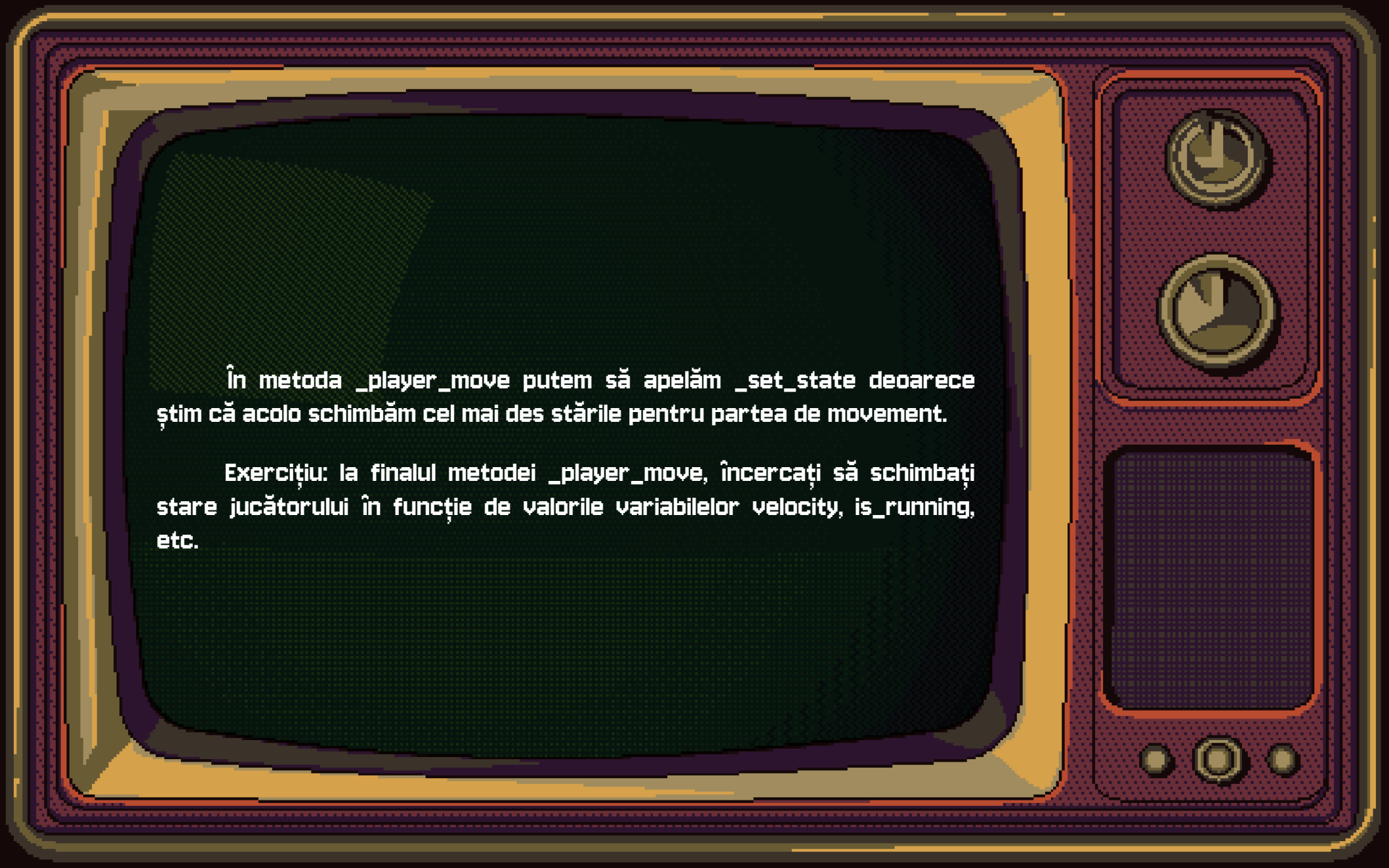
Exercițiu: Creați o metodă `_set_state` care primește ca parametru o variabilă de tip `state`(puteți să numiți variabila `'new_state'`) și returnează `void`.

În corpul metodei, setați variabila `_current_state` creată anterior la variabila primită ca parametru `new_state`.

Creați un `match statement` care compară `current state-ul` și faceți un `branch` pentru fiecare stare pe care o avem în `enum-ul` de stări.

În `branch-uri` printați starea curentă într-un `string`.

```
▼ func _set_state(new_state: State) -> void:
  >| _current_state = new_state
  >|
  ▼ >| match _current_state:
  ▼ >| >| State.IDLE:
  >| >| >| print("idle")
  ▼ >| >| State.WALKING:
  >| >| >| print("walking")
  ▼ >| >| State.RUNNING:
  >| >| >| print("running")
  ▼ >| >| State.USING_TOOL:
  >| >| >| print("using tool")
  ▼ >| >| _:
  >| >| >| print("state not found!")
```

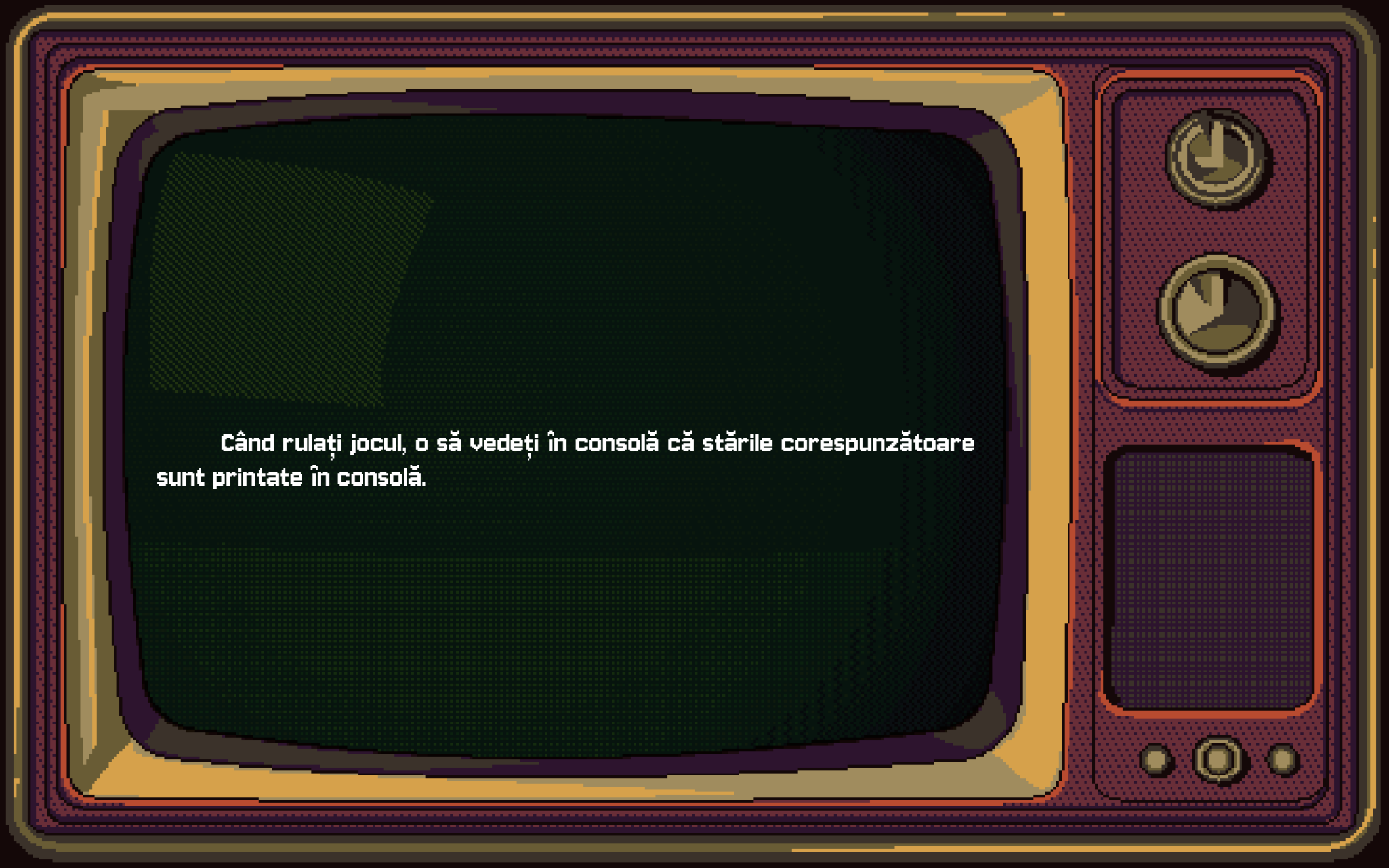


În metoda `_player_move` putem să apelăm `_set_state` deoarece știm că acolo schimbăm cel mai des stările pentru partea de movement.

Exercițiu: la finalul metodei `_player_move`, încercați să schimbați stare jucătorului în funcție de valorile variabilelor `velocity`, `is_running`, etc.



```
✓ >| if velocity == Vector2.ZERO:  
    >| >| _set_state(State.IDLE)  
✓ >| elif is_running:  
    >| >| _set_state(State.RUNNING)  
✓ >| else:  
    >| >| _set_state(State.WALKING)
```



Când rulați jocul, o să vedeți în consolă că stările corespunzătoare sunt printate în consolă.

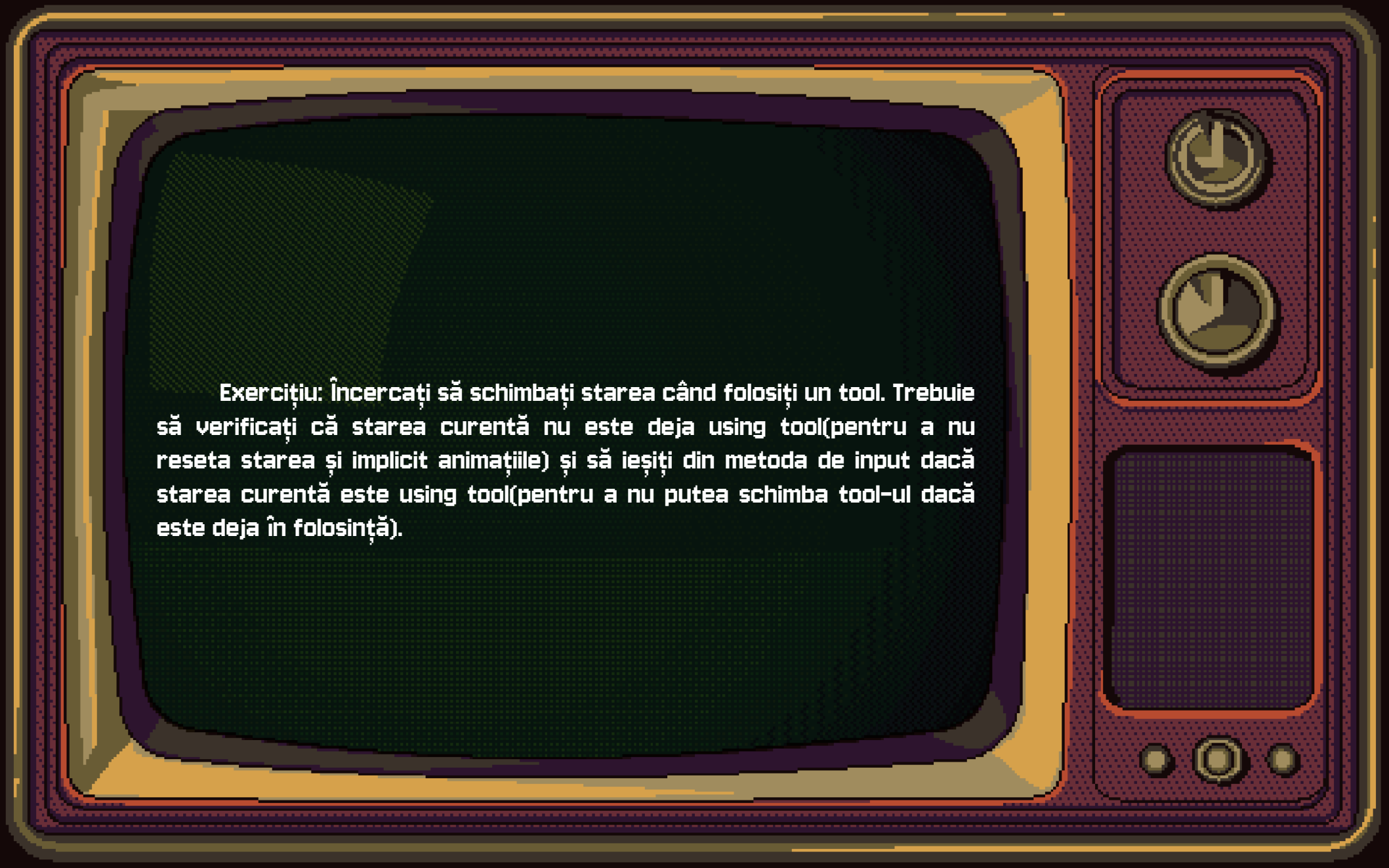
Exercițiu: modificați metoda `_player_animate` să primească o variabilă `anim_name` ca parametru de tipul `String`.

În corpul metodei apelați `play` pe ambele animation players și dați ca parametru `anim name`.

În metoda `set state` apelați metoda `_player_animate` pentru fiecare stare și dați numele animației corespunzătoare stării și metoda `_play_tool_animation` pentru starea de `using tool`.

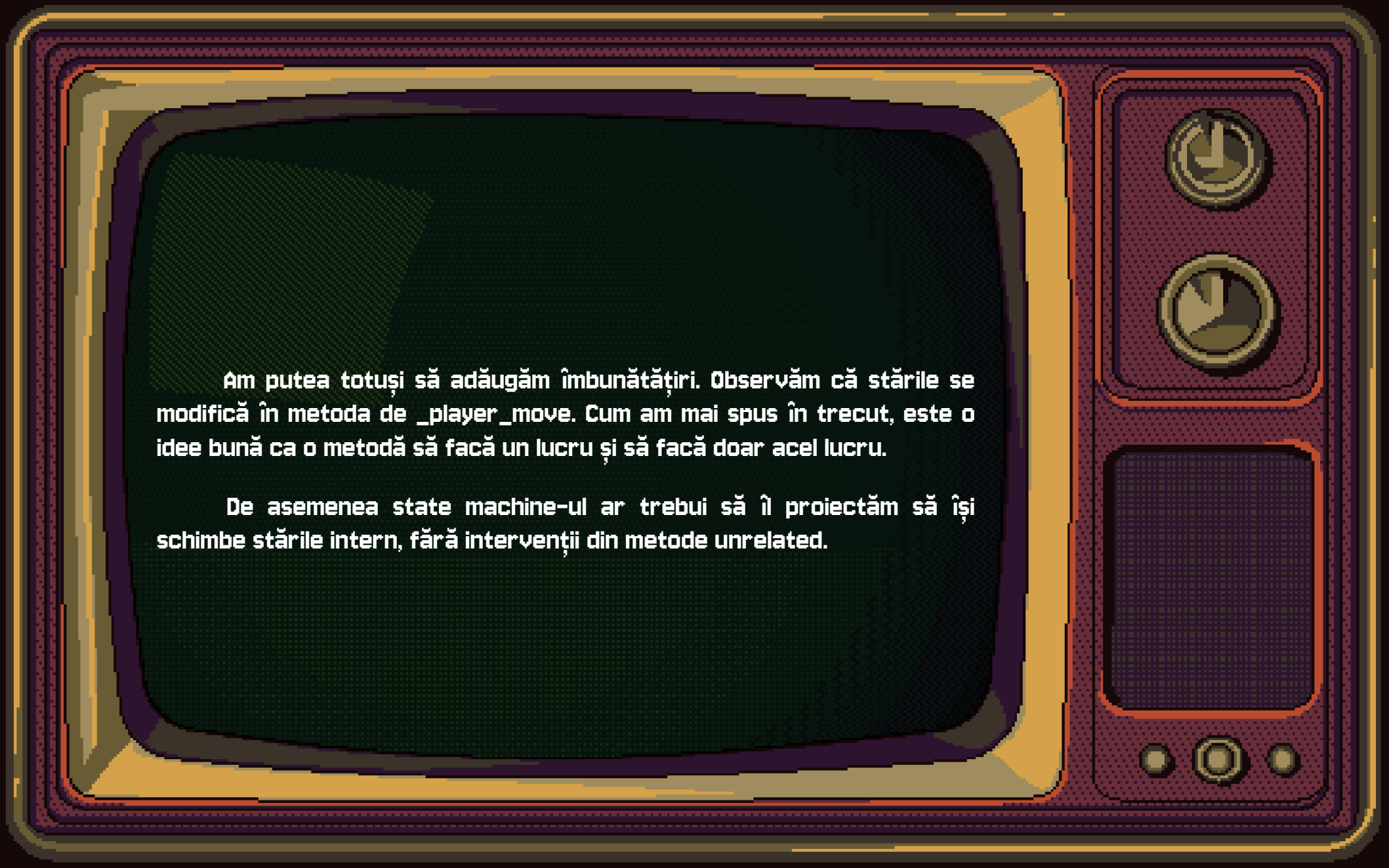

```
▼ func _set_state(new_state: State) -> void:
  >I _current_state = new_state
  >I
  ▼ >I match _current_state:
  ▼ >I >I State.IDLE:
  >I >I >I _player_animate("idle")
  ▼ >I >I State.WALKING:
  >I >I >I _player_animate("walk")
  ▼ >I >I State.RUNNING:
  >I >I >I _player_animate("run")
  ▼ >I >I State.USING_TOOL:
  >I >I >I _play_tool_animation()
  ▼ >I >I _:
  >I >I >I print("state not found!")

▼ func _player_animate(anim_name: String) -> void:
  >I character_animated_sprite.play(anim_name)
  >I tools_animated_sprite.play(anim_name)
```



Exercițiu: Încercați să schimbați starea când folosiți un tool. Trebuie să verificați că starea curentă nu este deja using tool(pentru a nu reseta starea și implicit animațiile) și să ieșiți din metoda de input dacă starea curentă este using tool(pentru a nu putea schimba tool-ul dacă este deja în folosință).

```
>| if Input.is_action_just_pressed("click") and _current_state != State.USING_TOOL:  
>| >| _set_state(State.USING_TOOL)  
>|  
>| # don't allow switch if already using toolad  
>| if _current_state == State.USING_TOOL:  
>| >| return  
>| if Input.is_action_pressed("next_tool"):  
>| >| _switch_tool(1)  
>| elif Input.is_action_pressed("previous_tool"):  
>| >| _switch_tool(-1)
```



Am putea totuși să adăugăm îmbunătățiri. Observăm că stările se modifică în metoda de `_player_move`. Cum am mai spus în trecut, este o idee bună ca o metodă să facă un lucru și să facă doar acel lucru.

De asemenea state machine-ul ar trebui să îl proiectăm să își schimbe stările intern, fără intervenții din metode unrelated.



Exercițiu: În metoda `_set_state`, dacă `current state` este egal cu `new_state`, faceți un `early return` din metodă (nu mai trebuie să verificăm în metoda de input că folosim deja un tool).

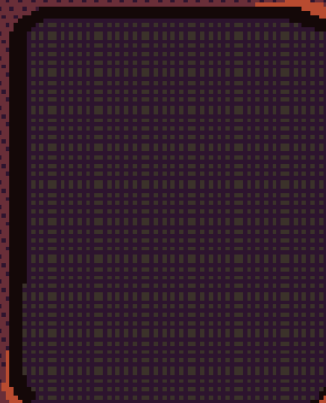
```
▼ func _set_state(new_state: State) -> void:
▼ >| if _current_state == new_state:
>| >| return
>| _current_state = new_state
>|
▼ >| match _current_state:
▼ >| >| State.IDLE:
>| >| >| character_animated_sprite.play("idle")
>| >| >| tools_animated_sprite.play("idle")
▼ >| >| State.WALKING:
>| >| >| character_animated_sprite.play("walk")
>| >| >| tools_animated_sprite.play("walk")
▼ >| >| State.RUNNING:
>| >| >| character_animated_sprite.play("run")
>| >| >| tools_animated_sprite.play("run")
▼ >| >| State.USING_TOOL:
>| >| >| _play_tool_animation()
```

Exercițiu: Creați o metodă `_update_state` care nu primește niciun parametru și returnează `void`.

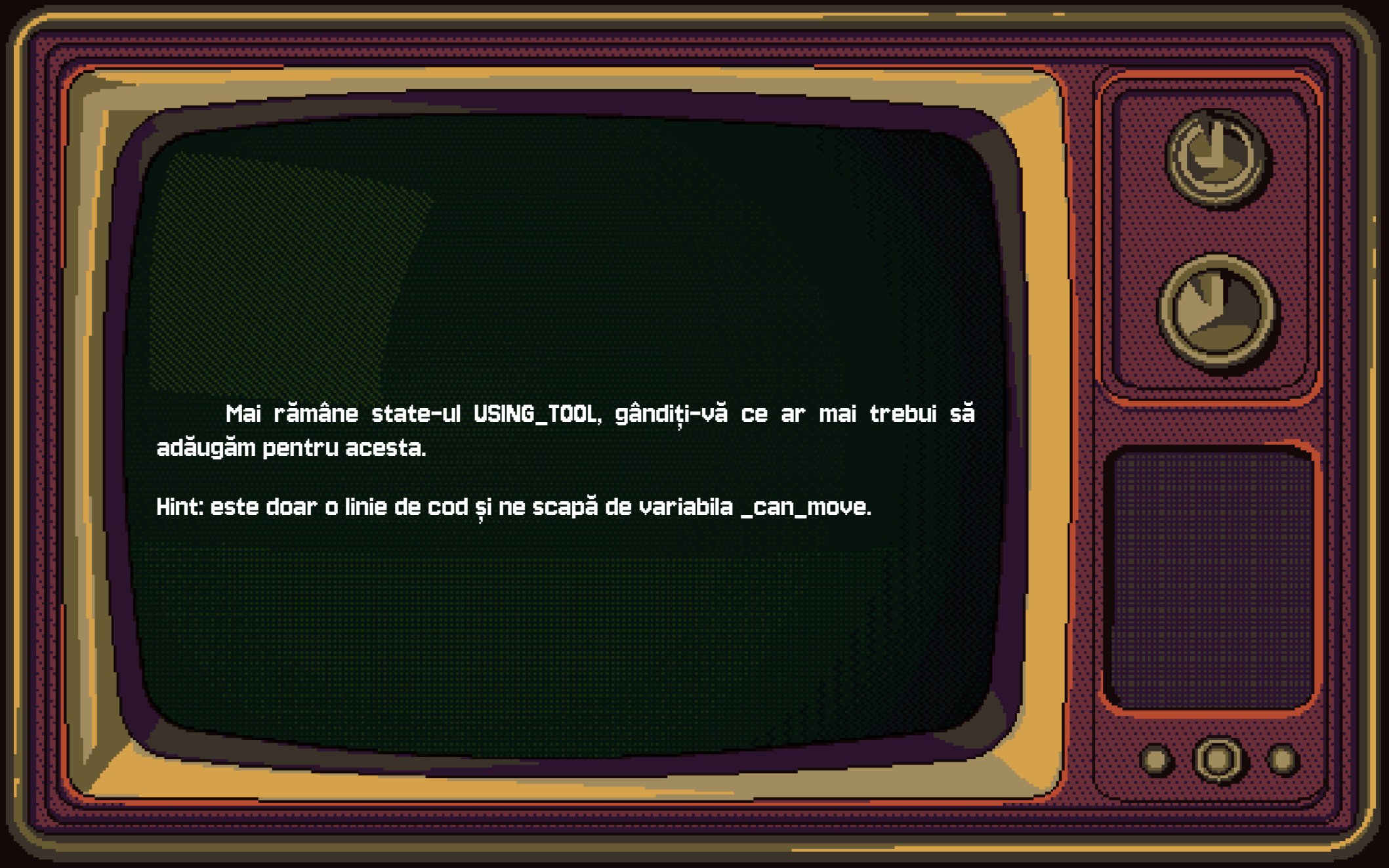
Creați un `match statement` folosindu-vă de variabila `_current_state`. Comparați cu state-urile pe care le avem, cum ați făcut și la `_set_state`. Apelați metoda din `physics_process` în locul metodei `_player_move`.

Hint: Majoritatea codului este scris, trebuie doar `copy paste` din alte metode în cazurile din `match statement`.

În case-urile pentru `match statement`-uri puteți avea mai multe variabile separate de virgulă.




```
▼ func _update_state() -> void:
▼ |   match _current_state:
▼ |     |   State.IDLE, State.RUNNING, State.WALKING:
|     |     |   _player_move()
▼ |     |     |   if velocity == Vector2.ZERO:
|     |     |     |   _set_state(State.IDLE)
▼ |     |     |   elif _is_running:
|     |     |     |   _set_state(State.RUNNING)
▼ |     |     |   else:
|     |     |     |   _set_state(State.WALKING)
|     |     |
▼ |     |     |   if Input.is_action_just_pressed("click"):
|     |     |     |   _set_state(State.USING_TOOL)
```

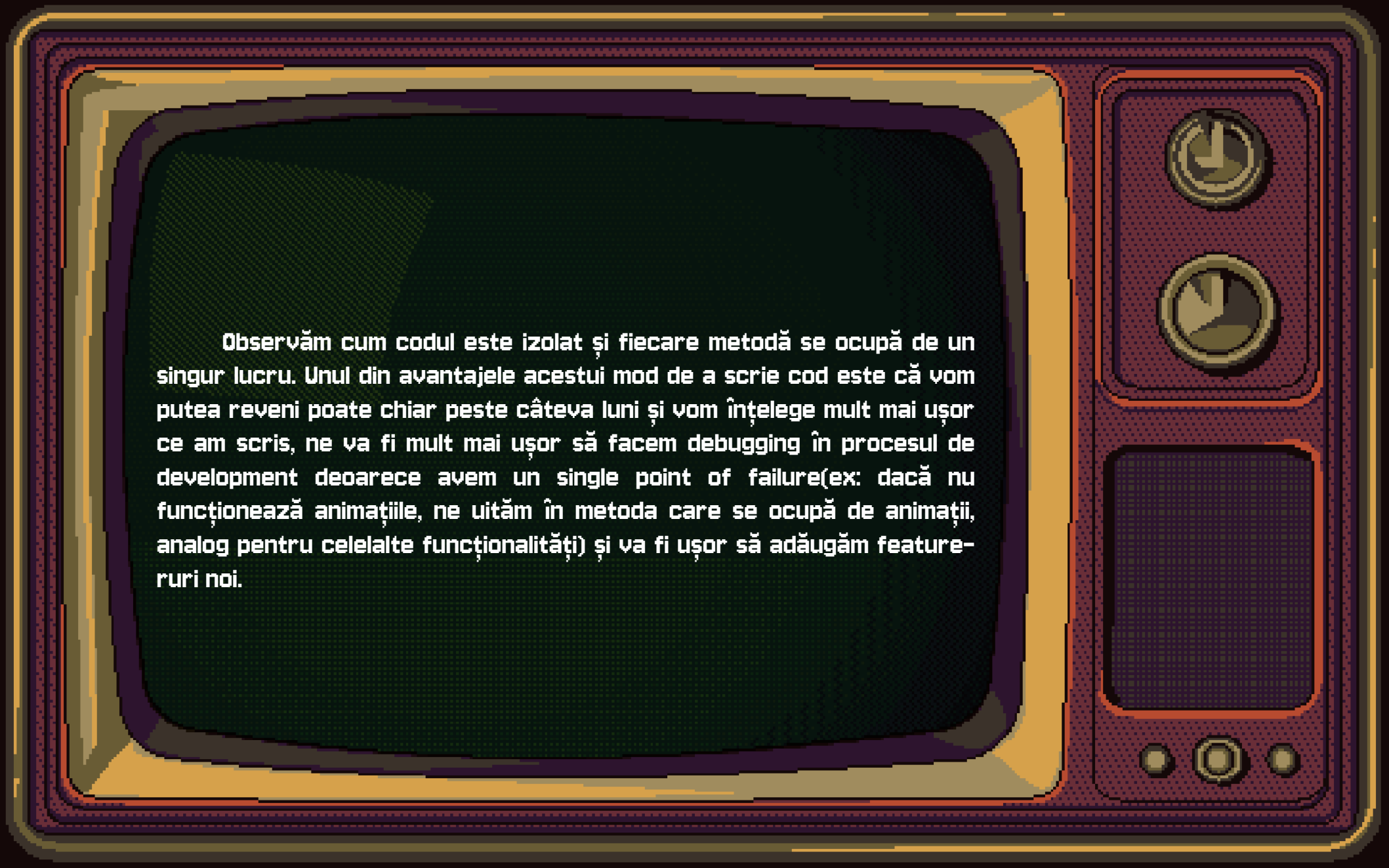


Mai rămâne state-ul `USING_TOOL`, gândiți-vă ce ar mai trebui să adăugăm pentru acesta.

Hint: este doar o linie de cod și ne scapă de variabila `_can_move`.

```
▼ func _update_state() -> void:
▼ | match _current_state:
▼ | | State.IDLE, State.RUNNING, State.WALKING:
| | | _player_move()
▼ | | | if velocity == Vector2.ZERO:
| | | | _set_state(State.IDLE)
▼ | | | elif _is_running:
| | | | _set_state(State.RUNNING)
▼ | | | else:
| | | | _set_state(State.WALKING)
| | |
▼ | | | if Input.is_action_just_pressed("click"):
| | | | _set_state(State.USING_TOOL)
▼ | | State.USING_TOOL:|
| | | velocity = Vector2.ZERO
```

```
▼ func _play_tool_animation() -> void:
    » # error handling demonstration
    » var _tool_anim: String = _tool_anims.get(_current_tool, "")
    » character_animated_sprite.play(_tool_anim)
    » tools_animated_sprite.play(_tool_anim)
    »
    » # wait for animation to finish to reset state
    » await character_animated_sprite.animation_finished
    » await tools_animated_sprite.animation_finished
    »
    » tool_used.emit(_current_tool)
    » # go back to the initial states
    » _set_state(State.IDLE)
```

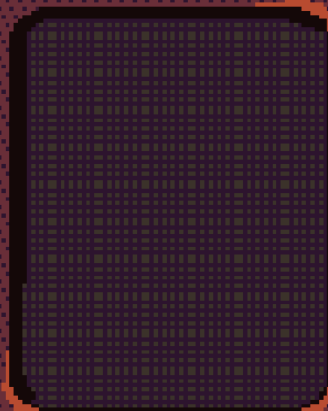


Observăm cum codul este izolat și fiecare metodă se ocupă de un singur lucru. Unul din avantajele acestui mod de a scrie cod este că vom putea reveni poate chiar peste câteva luni și vom înțelege mult mai ușor ce am scris, ne va fi mult mai ușor să facem debugging în procesul de development deoarece avem un single point of failure(ex: dacă nu funcționează animațiile, ne uităm în metoda care se ocupă de animații, analog pentru celelalte funcționalități) și va fi ușor să adăugăm feature-uri noi.

Să revenim la implementarea tool-urilor

În acest moment putem să dăm click, player-ul se va anima și va emite un semnal către nivel.

Urmează să modificăm terenul în funcție de tool-ul folosit de actor.



Să gândim pas cu pas ce vrem să construim în continuare.

Ce credeți că am avea nevoie în primul rând ca să interacționăm cu terenul?

- Când folosim tool-ul, să trimitem și poziția din fața jucătorului
- Coordonatele terenului (știm că terenul este un tilemap, adică o matrice cu două coordonate x, y)

The image shows a pixelated, retro-style interface. On the left is a large, dark, rounded rectangular area that serves as a main display or workspace. To the right of this area is a vertical control panel. At the top of the panel are two circular buttons, each with a small icon inside. Below these is a larger rectangular area with a grid pattern, possibly a secondary display or a menu. At the bottom of the panel are three small circular buttons. The entire interface is framed by a thick, multi-layered border with a yellow and orange color scheme.

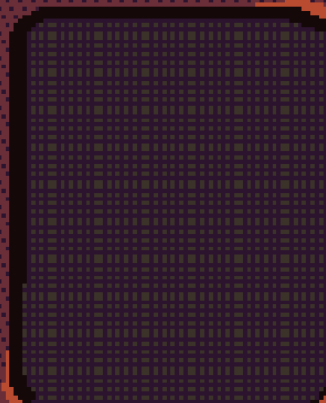
Avem două metode de a lua poziția tool-ului când interacționează
cu terenul, o metodă matematică și o metodă care evită complet
matematic.

Metoda matematică

În primul rând, la semnalul creat în `player.gd`, adăugați o variabilă numită `offset` de tip `Vector2`.

În metoda `_play_tool_animation`, înainte să emitem semnalul de `tool_used`, trebuie să calculăm o poziție ideal în fața jucătorului.

Exercițiu: Creați o variabilă de tip `Vector2` și inițializați variabila cu poziția jucătorului + un `Vector2` cu o valoare pe axa x, de exemplu 12.





The image shows a pixel-art style television set. The screen is dark and displays two lines of code. The control panel on the right side of the television features two circular knobs at the top, a rectangular speaker grille in the middle, and three small circular buttons at the bottom. The entire television is rendered in a retro, low-resolution aesthetic.

```
var pos_offset: Vector2 = position + Vector2(12, 0.0)  
tool_used.emit(_current_tool, pos_offset)|
```

Totuși, soluția noastră ține cont doar de o direcție, trebuie să ținem cont și de cealaltă direcție.

Adăugați o variabilă numită `tool_dir` de tip `float` și înmulțiți-o cu numărul 12 din `Vector2`.

Hint: pentru a asigna `tool_dir` puteți să vă folosiți de variabila `_facing_right`.



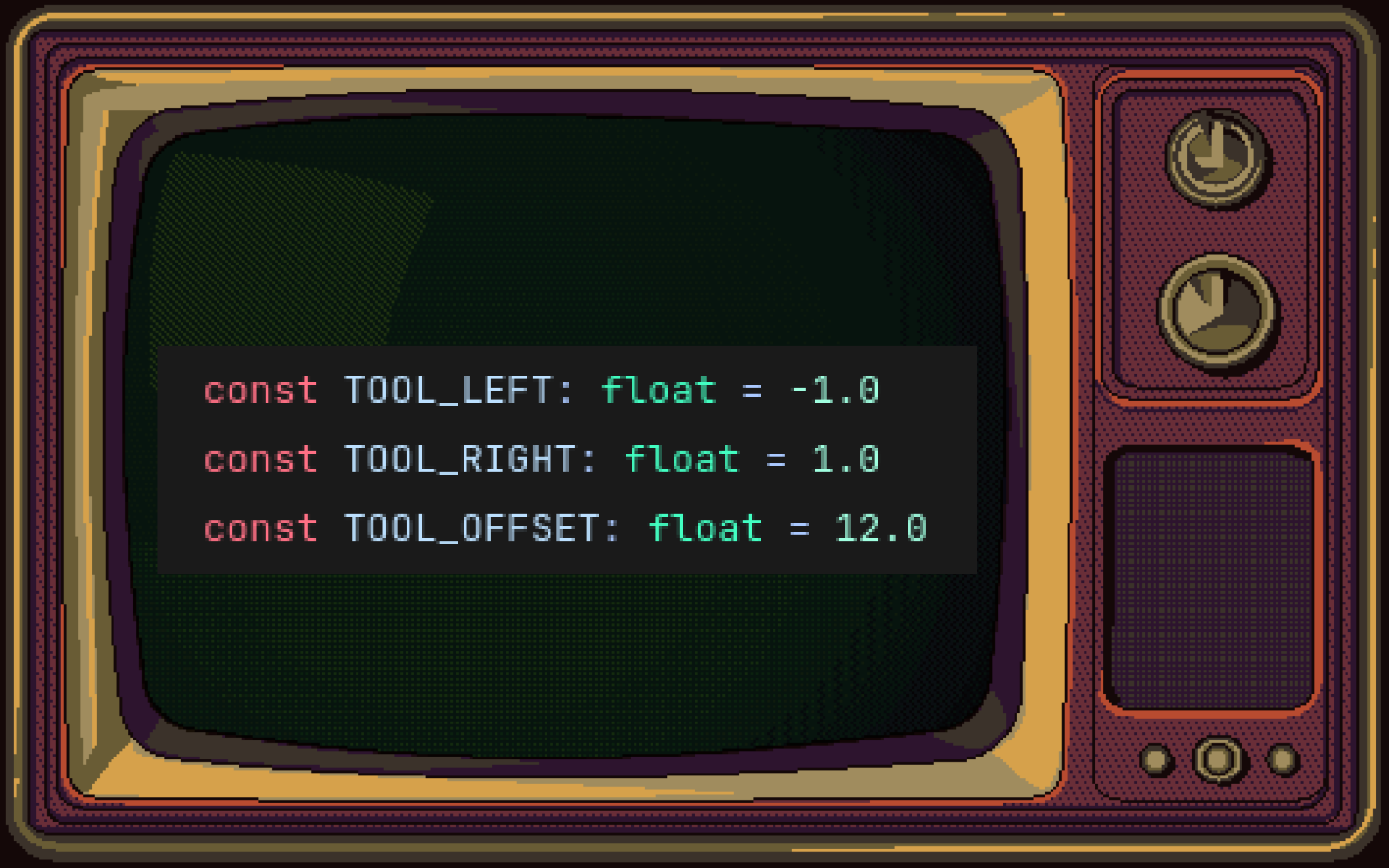
The image shows a pixel-art style television set. The screen is dark and displays three lines of code. To the right of the screen is a control panel with two circular knobs at the top, a rectangular speaker grille in the middle, and three small circular buttons at the bottom. The entire television is framed by a thick, multi-layered border.

```
var tool_dir: float = 1.0 if _facing_right else -1.0  
var pos_offset: Vector2 = position + Vector2(tool_dir * 12.0, 0.0)  
tool_used.emit(_current_tool, pos_offset)
```

În general este bine să nu lăsăm 'numere magice' în cod.

Exercițiu: Adăugați 3 variabile const de tip float și numiți-le TOOL_LEFT, TOOL_RIGHT și TOOL_OFFSET. Inițializați variabilele cu valorile -1.0, 1.0 și 12.0.

Variabilele de tip const sunt variabile pe care nu le puteți modifica la runtime.



```
const TOOL_LEFT: float = -1.0  
const TOOL_RIGHT: float = 1.0  
const TOOL_OFFSET: float = 12.0
```



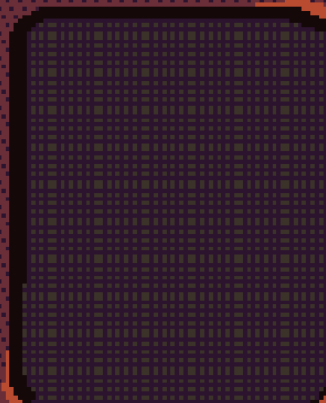

```
>| var tool_dir: float = TOOL_RIGHT if _facing_right else TOOL_LEFT
>| var pos_offset: Vector2 = position + Vector2(tool_dir * TOOL_OFFSET, 0.0)
>| tool_used.emit(_current_tool, pos_offset)
```

Acum că avem poziția tool-ului calculată, trebuie să o translatăm în coordonatele de pe grid.

Exercițiu: În metoda care răspunde la semnalul de tool used din scriptul level.gd, printați poziția pe care o primiți în metodă. Nu uitați să updatați metoda din level să primească ca parametru și o poziție de tip Vector2.

După ce ați printat poziția, încercați să creați o variabilă numită grid_pos de tip Vector2i(un vector de numere întregi) și gândiți ce coordonate ar trebui să aibă.

Hint: aveți nevoie de mărimea tile-urilor din tilemap care știm că este 16.

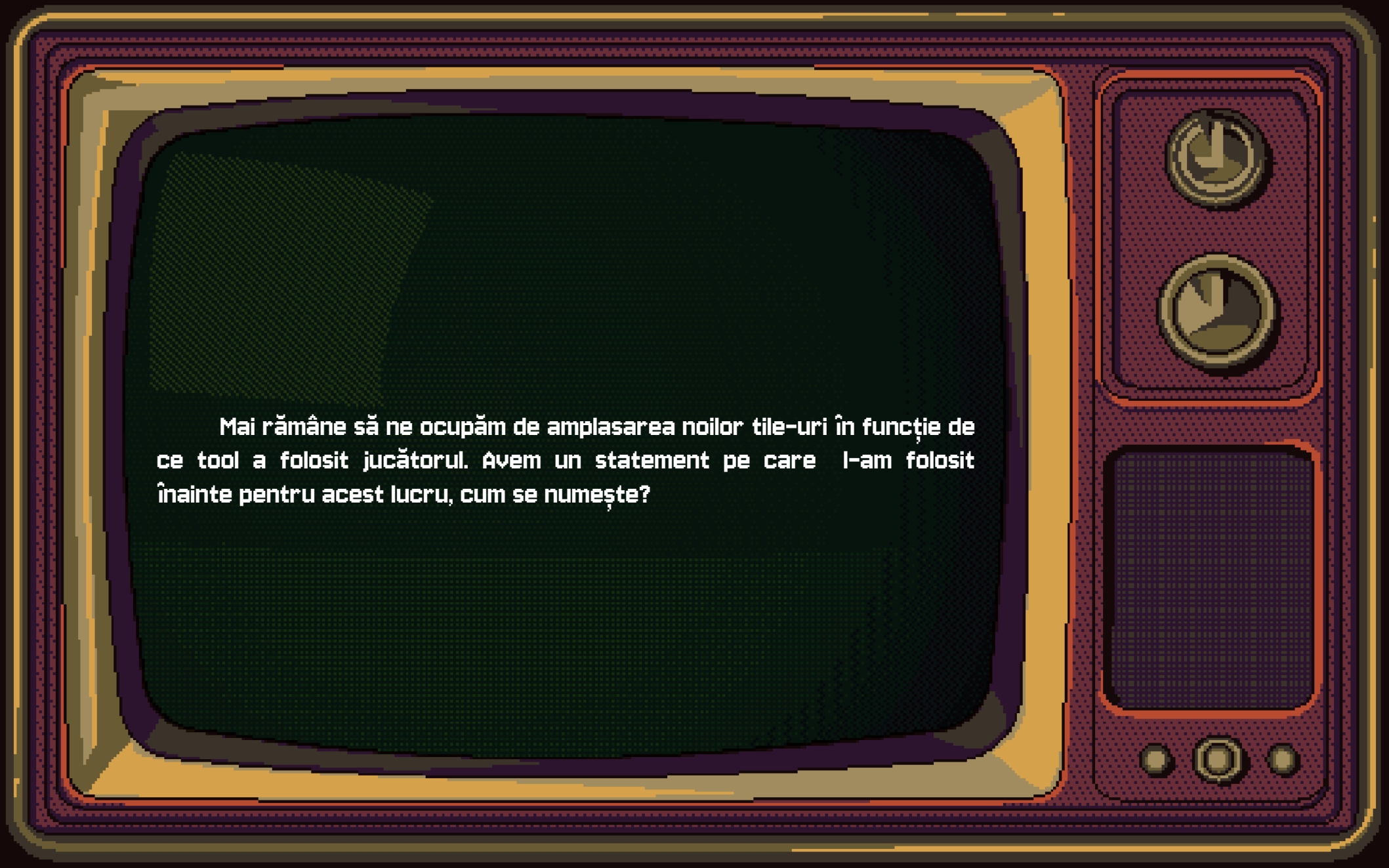


Creați o variabilă const numită `TILE_SIZE` de tip `int` pentru a elimina numărul magic 16

```
▼ func _on_player_tool_used(tool: PlayerController.Tool, pos: Vector2) -> void:  
  >| var grid_pos: Vector2i = Vector2i(int(pos.x / 16), int(pos.y / 16))
```



```
▼ func _on_player_tool_used(tool: PlayerController.Tool, pos: Vector2) -> void:  
    >| var grid_pos: Vector2i = Vector2i(int(pos.x / TILE_SIZE), int(pos.y / TILE_SIZE))
```

The image shows a pixelated, retro-style interface. On the left is a large, dark, rounded rectangular area that serves as a main display or workspace. To the right of this area is a vertical control panel. At the top of the panel are two circular buttons, each with a small icon inside. Below these is a larger rectangular area with a grid pattern, possibly a secondary display or a menu. At the bottom of the panel are three small circular buttons. The entire interface is framed by a thick, multi-layered border with a yellow and orange color scheme.

Mai rămâne să ne ocupăm de amplasarea noilor tile-uri în funcție de ce tool a folosit jucătorul. Avem un statement pe care l-am folosit înainte pentru acest lucru, cum se numește?

Folosiți un match statement de data aceasta știind tool-ul. Pentru cazuri adăugați cazul când folosim lopata.

Hint: Adăugați cuvântul cheie `class_name` și dați un nume clasei 'Player' în scriptul Player.

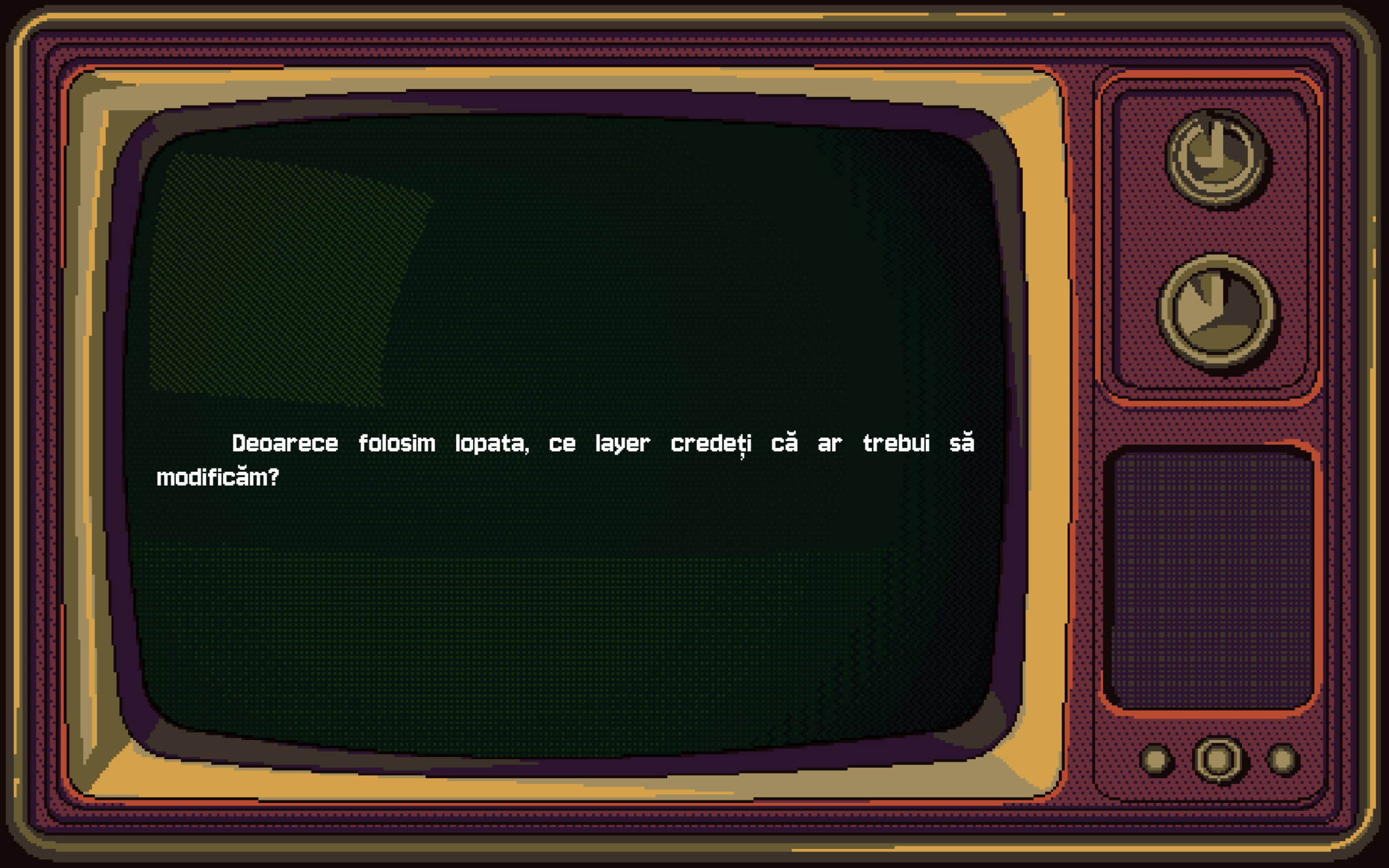
După ce am dat un nume clasei player vom putea accesa enum-ul tool din scriptul level.gd: `Player.Tool.SHOVEL`. Printați string-ul 'SHOVEL'.

player_controller.gd

```
1  class_name Player extends CharacterBody2D
2
```



```
→ 27 func _on_player_tool_used(tool: Player.Tool, pos: Vector2) -> void:
    28     var grid_pos: Vector2i = Vector2i(int(pos.x / TILE_SIZE), int(pos.y / TILE_SIZE))
    29     match tool:
    30     | Player.Tool.SHOVEL:|
    31     |>|>| print("shovel")
```

Deoarece folosim lopata, ce layer credeți că ar trebui să modificăm?



Știind că trebuie să modificăm layer-ul de dirt. Adăugați o referință la acel layer în script(puteți dând click și CTRL+drag and drop în script)

```
7 @onready var dirt_layer: TileMapLayer = %DirtLayer|
```

DirtLayer are tipul **TileMapLayer**, dacă nu știm ce metode să folosim putem accesa documentația godot pentru TileMapLayer și vom vedea toate metodele folosibile pe TileMapLayers.

Avem poziția de pe hartă unde vrem să amplasăm un tile din cod. Pentru acest lucru, clasa TileMapLayer expune mai multe metode, noi vom folosi metoda **set_cell** momentan.

Exercițiu: citiți în documentația Godot și încercați să înțelegeți cum să o folosiți pe dirt_layer.

```
▼ func _on_player_tool_used(tool: Player.Tool, pos: Vector2) -> void:
  >|   var grid_pos: Vector2i = Vector2i(int(pos.x / TILE_SIZE), int(pos.y / TILE_SIZE))
▼ >|   match tool:
▼ >|     >|   Player.Tool.SHOVEL:
  >|     >|     >|   print("shovel")
  >|     >|     >|   dirt_layer.set_cell(grid_pos, 0, Vector2i(6, 35))
```

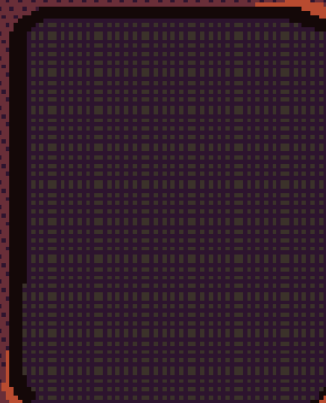
Dacă rulăm jocul, când folosim lopata o să putem săpa în pământ.

Observăm totuși o problemă în cod, folosim un număr magic `Vector2`.

Exercițiu: Creați un enum cu numele `CellType` și adăugați momentan o constantă `DIRT = 0`

Creați o variabilă `@export` numită `_atlas_coordinates` de tip `Dictionary` cu setul cheie-valoare `[CellType, Vector2i]`. Din editor vom putea seta ce tile din atlas îi corespunde fiecărui tip de teren.

Selectați nodul `level` și în inspector veți vedea variabila `atlas coordinates` unde puteți adăuga tipul celulei(tile-ului) și coordonata din atlas care se potrivește.





```
enum CellType {  
    DIRT = 0  
}  
  
@export var _atlas_coordinates: Dictionary[CellType, Vector2i]
```

Level

Filter Properties

LevelManager

Atlas Coordinat ↺

Dictionary[int, Vec

Dirt

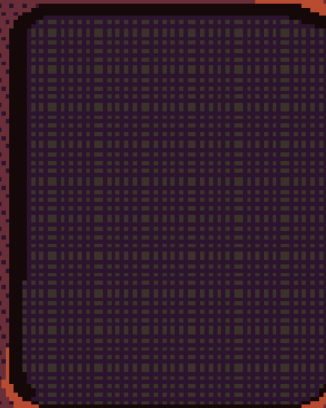
x 6

y 35

New Key: Dirt

New Value: x 0 y 0

+ Add Key/Value Pair



Temă(ABSOLUT OPȚIONALĂ):

- Încercați să mai adăugați tool-uri și funcționalități pentru ele
- La dirt layer putem să folosim o metodă numită `set_cells_terrain_connect` împreună cu funcționalitatea de autotiling.



Thank you for your attention!